

Потрага за благом

Временско ограничење: 8 s Ограничење меморије: 256 MiB

Тражиш давно изгубљено благо на огромном $N \times N$ пољу ћелија помоћу магичног компаса. Укупно постоји $K \leq 3$ сандука са благом сакривених у различитим ћелијама поља. Твој циљ је једноставан: пронаћи их све!

Када поставиш компас на једну од ћелија, он ће пронаћи најкраће путеве до сандука са благом користећи само потезе лево, горе, десно и доле (тј. до најближег сандука са благом по Менхетн растојању). Затим ће показати смер првог потеза. Ако више исправних првих потеза води до најближег сандука са благом (или до више њих), компас ће вратити све њих. Ако ћелија садржи благо, компас ће то назначити.

Кад год пронађеш сандук са благом, испразниш његов садржај, али не можеш да уклониш сандук – превише је тежак. Твој компас не зна да ли је сандук пун или празан. Он ће показивати пут до најближег сандука, празног или пуног.

Покушај да пронађеш локације свих сандука са благом уз што мањи број упита компасу.

Задатак

Ово је интерактиван задатак. У сваком тест примеру (тј. при сваком извршавању твог програма), твој програм ће морати да реши неколико потрага за благом. Са оцењивачем треба да комуницираш користећи библиотеку коју обезбеђују организатори. Ова библиотека садржи следеће декларације:

- **void** *NextHunt*(**int** &*N*, **int** &*K*) — позови ову функцију да започнеш следећу потрагу за благом. Она ће поставити величину мреже у променљиву *N* и број блага у *K*. Ако у текућем извршавању више нема потрага за благом које треба решити, функција ће поставити *N* и *K* на -1 ; у том случају треба да завршиш програм са излазним кодом 0. Имај у виду да ову функцију можеш позвати пре него што пронађеш сва блага у текућој потрази, нпр. ако покушаваш да решиш само за делимичне поене.
- **enum** {*TREASURE* = 0, *DIR_RIGHT* = 1, *DIR_UP* = 2, *DIR_LEFT* = 4, *DIR_DOWN* = 8 }; — ово су константе које се користе у повратним вредностима функције *Query* (види испод).
- **int** *Query*(**int** *x*, **int** *y*) — ако ћелија са координатама (*x*, *y*) садржи благо, ова функција враћа *TREASURE*. У супротном враћа збир једне или више константи *DIR_RIGHT*, *DIR_UP*, *DIR_LEFT* и *DIR_DOWN*, означавајући које потезе компас враћа када се постави на ћелију (*x*, *y*). Координате *x* и *y* морају бити цели бројеви од 0 до $N - 1$. (Напомена: у овом задатку, *y*-координате расту одозго надоле.)

Након што *NextHunt* постави $N = K = -1$, твој програм не сме поново да позове ни *NextHunt* ни *Query*. Такође не сме да позове *Query* пре првог позива функције *NextHunt*. Ако твој програм не поштује ова ограничења, или ако направи више од

1000 упита у оквиру једне потраге за благом, или ако при позиву функције *Query* проследи вредности x и/или y ван дозвољеног опсега, библиотека ће прекинути твој програм и вратити пресуду `run-time-error` (RTE) за текући тест пример.

Сматра се да је благо пронађено ако си бар једном упитао ћелију која га садржи. Ако твој програм позове *NextHunt* пре него што пронађе сва блага у текућој потрази, то се не сматра грешком, али ће утицати на твој резултат (више о бодовању испод).

Да би приступио библиотеци, твој програм треба да укључи заглавље `treasurehuntlib.h`:

```
#include "treasurehuntlib.h"
```

Ово заглавље можеш преузети овде: `treasurehuntlib.h`.

Као помоћ при развоју решења, једноставна имплементација библиотеке доступна је овде: `treasurehuntlib-public.cpp`. Да би је компајлирао заједно са својим програмом, само додај њено име као параметар компајлеру, нпр.:

```
g++ foo.cpp treasurehuntlib-public.cpp
```

ако је `foo.cpp` име фајла који садржи твоје решење.

Имплементација у `treasurehuntlib-public.cpp` подржава, поред горе наведених декларација, још једну функцију звану `void InitFromFile(const char *fileName)`, која чита списак потрага за благом из фајла и омогућава ти да играш игру са њима уместо да генерише сопствене насумичне потраге за благом. Више детаља ћеш наћи у самом фајлу `treasurehuntlib-public.cpp`.

На евалуационом серверу ће се користити друга имплементација библиотеке, тако да не треба да правиш никакве претпоставке о томе како тачно имплементација ради. Можеш, међутим, претпоставити да она не загађује глобални именски простор никаквим декларацијама осим оних које су горе наведене (*NextHunt*, *Query* и пет константи).

Твој код не сме да чита са стандардног улаза нити да пише на стандардни излаз, јер ће их наша имплементација библиотеке на евалуационом серверу користити за комуникацију са остатком евалуационог окружења.

Улаз

Ограничења

- $2 \leq N \leq 10^6$
- $1 \leq K \leq 3$
- У оквиру једног извршавања твог програма биће највише 100 000 потрага за благом.
- У оквиру једне потраге за благом можеш направити највише 1000 упита.
- Систем за оцењивање није адаптиван.

Подзадаци

- Подзатак 1 (10 поена) $K = 1$
- Подзатак 2 (30 поена) $K = 2$
- Подзатак 3 (60 поена) $K = 3$.

Бодовање

Подзадатак се може састојати од више тест примера (више извршавања твог програма), а сваки тест пример се може састојати од више потрага за благом. За потребе бодовања, све потраге за благом датог подзадатка оцењују се заједно, без обзира на то како су првобитно биле распоређене по тест примерима. За i -ту потрагу за благом, означимо величину мреже са $N_i \times N_i$, број блага са K_i , број упита које је направио твој програм са Q_i и број блага које је твој програм пронашао са F_i . Даље, означимо са S укупан број поена доступан за овај подзадатак. Тада је број поена који твој програм добија за овај подзадатак:

- Ако је твој програм увек пронашао сва блага (тј. ако је $F_i = K_i$ за све i), његов резултат зависи од $t_i = \frac{Q_i}{\lceil \log_2 N_i \rceil}$:

$$\frac{S}{2} + \frac{S}{2} \cdot \min_i f(t_i) \text{ поена, где је } f(t_i) = \begin{cases} 1, & t_i \leq 11 \\ 1 - (t_i - 11)/9, & 11 \leq t_i \leq 20 \\ 0, & t \geq 20. \end{cases}$$

- Ако твој програм није увек пронашао сва блага (тј. ако постоји i такво да је $F_i < K_i$), добија

$$\frac{S}{2} \cdot \min_i \frac{F_i}{K_i} \text{ поена.}$$

Другим речима, половину поена добијаш за проналажење свих блага, а другу половину за то да их пронађеш у што мање упита. За максималан број поена, твоје решење треба да пронађе сва блага у највише $11 \lceil \log_2 N_i \rceil$ упита. Између $11 \lceil \log_2 N_i \rceil$ и $20 \lceil \log_2 N_i \rceil$ упита, резултат се линеарно смањује; а ако твоје решење направи више од $20 \lceil \log_2 N_i \rceil$ упита, добиће само прву половину поена за проналажење свих блага. (Симболи $\lceil \cdot \rceil$ значе да се вредност $\log_2 N_i$ заокружује навише на најближи цео број.)

Ако формуле изнад дају нецео број поена за подзадатак, он ће бити заокружен на најближи цео број.

Ако твој програм има грешку у извршавању или се не придржава горе поменутог протокола при коришћењу библиотеке, добиће 0 поена за цео подзадатак. Да би добио делимичне поене за проналажење подскупа блага, твој програм зато мора да заврши претрагу коректно позивом функције *NextHunt*.

Пример

Позив	Повратна вредност
NextHunt(N, K)	$N = 4, K = 1$
Query(2, 0)	$DIR_DOWN + DIR_RIGHT = 9$
Query(3, 1)	DIR_DOWN
Query(3, 2)	$TREASURE$
NextHunt(N, K)	$N = -1, K = -1$